

COL100: Introduction to Computer Science

Lab 9: Tuples and Dictionary

March 17, 2025

1 Tuples

In Python, tuples are immutable ordered sequences of elements. Unlike strings, the elements of a tuple can be of any type, and they do not need to be of the same type as each other. Tuples are similar to lists, but unlike lists, they cannot be modified once created.

1.1 Creating Tuples

A tuple is created by enclosing a comma-separated list of elements in parentheses. Here are some examples:

```
1 t1 = ()           # an empty tuple
2 t2 = (1, 'two', 3) # a tuple containing three elements
```

When we print these tuples:

```
1 print(t1)
2 print(t2)
```

The output will be:

```
1 ()
2 (1, 'two', 3)
```

1.2 Singleton Tuple

You might expect that the tuple containing the single value 1 would be written as (1). However, parentheses are also used for grouping expressions. Therefore, (1) is just a way to write the integer 1. To create a tuple with a single element, you must add a comma after the element, like so:

```
1 singleton_tuple = (1,)
```

This defines a tuple with one element.

1.3 Repetition and Concatenation of Tuples

Tuples support repetition, similar to strings. For example:

```
1 repeated_tuple = 3 * ('a', 2)
```

This expression evaluates to:

```
1 ('a', 2, 'a', 2, 'a', 2)
```

Tuples can also be concatenated. For example, consider the following code:

```
1 t1 = (1, 'two', 3)
2 t2 = (t1, 3.25)
```

Now, when we print these tuples:

```
1 print(t2)
```

The output will be:

```
1 ((1, 'two', 3), 3.25)
```

Here, `t2` contains a tuple `t1` and a floating-point number 3.25. Since tuples can contain other tuples, this is perfectly valid.

1.4 Indexing and Slicing Tuples

You can index and slice tuples just like you can with strings or lists. Consider the following:

```
1 t1 = (1, 'two', 3)
2 t2 = (t1, 3.25)
3 print(t1 + t2)           # Concatenate t1 and t2
4 print((t1 + t2)[3])      # Access the fourth element
5 print((t1 + t2)[1:3])    # Slice the concatenated tuple
```

The first print statement will produce:

```
1 (1, 'two', 3, (1, 'two', 3), 3.25)
```

The second print statement will output the fourth element of the concatenated tuple:

```
1 (1, 'two', 3)
```

The third print statement will output a slice of the concatenated tuple:

```
1 ('two', 3, (1, 'two', 3))
```

2 Dictionary

Dictionaries or objects of type `dict`, are a fundamental data structure in Python. Unlike lists, which are indexed by integers, dictionaries are indexed using keys. Keys in a dictionary must be hashable objects, meaning they must have:

- A `__hash__` method that maps the object to an integer, and the value returned by `__hash__` does not change during the lifetime of the object.
- An `__eq__` method for comparing the object with other objects for equality.

All scalar immutable types in Python, such as integers, strings, and tuples (if all their elements are hashable), are hashable. However, mutable types like lists or dictionaries themselves are not hashable.

2.1 Creating, Accessing, and Altering Dictionaries

A dictionary is created by enclosing key-value pairs within curly braces (`{}`), where each pair consists of a key and its associated value, separated by a colon. For example:

```
1 month_numbers = {'Jan':1, 'Feb':2, 'Mar':3, 'Apr':4, 'May':5,
2                 1: 'Jan', 2: 'Feb', 3: 'Mar', 4: 'Apr', 5: 'May'}
```

To access values in a dictionary, we use keys. Here is an example:

```
1 print(month_numbers)
2 print('The third month is ' + month_numbers[3])
3 dist = month_numbers['Apr'] - month_numbers['Jan']
4 print('Apr and Jan are', dist, 'months apart')
```

This code produces the output:

```
1 {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 1: 'Jan',
2  2: 'Feb', 3: 'Mar', 4: 'Apr', 5: 'May'}
3 The third month is Mar
4 Apr and Jan are 3 months apart
```

Note that the entries in a dictionary cannot be accessed by an index. In the above example, `month_numbers[1]` refers to the value associated with the key 1.

To check if a key exists in a dictionary, use the `in` operator, e.g., `print('Jan' in month_numbers)` will output `True`.

Dictionaries are mutable. You can add new entries or modify existing ones:

```
1 month_numbers['June'] = 6
2 month_numbers['May'] = 'V'
```

2.2 Using Dictionaries for Translation

Dictionaries simplify writing programs that rely on mappings. For example, consider a simple translation program:

```
1 FtoE = {'bois': 'drink', 'rouge': 'red', 'vin': 'wine'}
2 sentence = 'Je bois du vin rouge.'
3 print(translate(sentence, FtoE, 'French to English'))
```

This code translates the sentence into English, producing:

```
1 I drink of wine red.
```

However, remember that dictionaries are mutable, and modifications to the dictionary can lead to unintended side effects. For instance:

```
1 FtoE['bois'] = 'wood'
2 print(translate('Je bois du vin rouge.', FtoE, 'French to English'))
```

This code will produce:

```
1 I wood of wine red.
```

2.3 Iterating Over Dictionaries

Python allows multiple ways to iterate over dictionary entries.

Iterating Over Keys

A for loop of the form `for k in d` iterates over the keys of the dictionary `d`. For example:

```
1 capitals = {'France': 'Paris', 'Italy': 'Rome', 'Japan': 'Kyoto'}
2 for key in capitals:
3     print('The capital of', key, 'is', capitals[key])
```

This produces:

```
1 The capital of France is Paris
2 The capital of Italy is Rome
3 The capital of Japan is Kyoto
```

Iterating Over Values

To iterate over values, use the `values()` method:

```
1 cities = []
2 for val in capitals.values():
3     cities.append(val)
4 print(cities, 'is a list of capital cities')
```

This prints:

```
1 ['Paris', 'Rome', 'Kyoto'] is a list of capital cities.
```

Iterating Over Key/Value Pairs

To iterate over both keys and values simultaneously, use the `items()` method:

```
1 for key, val in capitals.items():
2     print(val, 'is the capital of', key)
```

This produces:

```
1 Paris is the capital of France
2 Rome is the capital of Italy
3 Kyoto is the capital of Japan
```

2.4 View Objects in Dictionaries

The `keys()`, `values()`, and `items()` methods return *view objects*. These are dynamic views of the dictionary that reflect any changes to the dictionary. For example:

```
1 cap_vals = capitals.values()
2 print(cap_vals)
3 capitals['Japan'] = 'Tokyo'
4 print(cap_vals)
```

This prints:

```
1 dict_values(['Paris', 'Rome', 'Kyoto'])
2 dict_values(['Paris', 'Rome', 'Tokyo'])
```

View objects can be converted to lists for easier manipulation. For example:

```
1 list(capitals.values())
```

2.5 Using Tuples as Keys

Tuples, being immutable and hashable, can be used as keys in dictionaries. For example, consider a tuple representing an airline flight as `(flight_number, day)`. Such tuples can easily be used as keys to store additional information:

```
1 flights = {('AA101', 'Monday'): '10:30 AM', ('BA202', 'Tuesday'): '1:00 PM'}
2 print(flights[('AA101', 'Monday')])
```

This prints:

```
1 10:30 AM
```

Lists, in contrast, cannot be used as keys because they are mutable and not hashable.

2.6 Efficiency of Dictionaries

While it is possible to implement a dictionary-like structure using lists of tuples, such an implementation is computationally inefficient. For example:

```
1 def key_search(L, k):
2     for elem in L:
3         if elem == k:
4             return 'Found'
5     return 'Not Found'
```

In this implementation, retrieving a value may require scanning the entire list, resulting in poor performance. Python's built-in dictionaries use hashing to provide nearly constant time lookup regardless of dictionary size.

3 Submission Problem

3.1 Text Processing for Restaurant Reviews

Create a program that processes restaurant reviews to perform sentiment analysis. Your program should extract valuable insights from customer feedback to help restaurant owners improve their service.

We know the reviewers use words like "tasty", "delicious", and "excellent" to express positive sentiments. Your task is to write a Python program that takes the following phrases in to account and calculates the sentiment score for the reviews.

Phrase	Score
amazing	2
great	2
delicious	2
slow	-1
terrible	-2
authentic	1

Table 1: List of Phrases that appear in reviews

Instructions

- The first line contains N, the number of reviews that need to be processed.
- The second line onwards contains the N reviews.
- Only the following punctuation marks are allowed: ', . ? ; " -

Input

```
1 3
2 The pasta at Luigi's was amazing, but the service was slow.
3 Great atmosphere at Luigi's. The desserts were delicious.
4 Luigi's has terrible parking options, though the food was amazing.
```

Output

```
1 5
```

Explanation: The following keywords appear in the input:

amazing (2), slow (-1), great (2), delicious (2), terrible (-2), amazing (2)
When adding up the scores, we get 5.

4 Practice Problems

4.1 Square of Number

4.1.1 Problem Statement

Write a function that takes a list of numbers and returns a tuple containing the square of each number.

4.1.2 Test Case 1

Input

```
1 [1, 2, 3, 4]
```

Output

```
1 (1, 4, 9, 16)
```

Explanation Each number in the list is squared, and the resulting squares are packed into a tuple.

4.1.3 Test Case 2

Input

```
1 [0, 5, 10]
```

Output

```
1 (0, 25, 100)
```

Explanation The square of 0 is 0, the square of 5 is 25, and the square of 10 is 100. These values are returned as a tuple.

4.2 Frequency Checker

4.2.1 Problem Statement

Write a function that takes a string and returns a dictionary mapping each character to its frequency.

4.2.2 Test Case 1

Input

```
1 "hello"
```

Output

```
1 {'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

Explanation The character “h” occurs once, “e” occurs once, “l” occurs twice, and “o” occurs once.

4.2.3 Test Case 2

Input

```
1 "banana"
```

Output

```
1 {'b': 1, 'a': 3, 'n': 2}
```

Explanation The character “b” occurs once, “a” occurs three times, and “n” occurs twice.

4.3 Anagram Problem

4.3.1 Problem Statement

Write a function that groups a list of words into a dictionary where the keys are sorted letters and the values are lists of anagrams.

4.3.2 Test Case 1

Input

```
1 ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
```

Output

```
1 {'aet': ['eat', 'tea', 'ate'], 'ant': ['tan', 'nat'], 'abt': ['bat']}]
```

Explanation Words that are anagrams are grouped under keys that represent their sorted letters.

4.3.3 Test Case 2

Input

```
1 ['listen', 'silent', 'enlist', 'rat', 'tar', 'art']
```

Output

```
1 {'eilnst': ['listen', 'silent', 'enlist'], 'art': ['rat', 'tar', 'art']}]
```

Explanation Anagrams such as “listen”, “silent”, and “enlist” are grouped together under the key “eilnst”. Similarly, “rat”, “tar”, and “art” are grouped under “art”.